Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Implementing queues with deque

Today
Welcome to your next lesson: Implementing queues with deque.

I'm getting your personalized lesson ready — based on what you've learned, what's ahead, and how you learn best.

Step by step, you’re moving forward.

Ron, let’s plug deque into what you already know. ⏳

Imagine a line of players waiting to join a game lobby. First in line should be the first to get in — that’s FIFO (First In, First Out).

In Python, we can model that line using deque as a queue:

We’ll use a deque object as our queue, and then use append to add to the back, and popleft to remove from the front.

Step by step, the first thing is just:

we need to import and create an empty deque.

Here’s the tiny goal for this step:

Import deque from collections
Create an empty queue

Visually:

Text

```
[  ]   ← empty queue (no players yet)
```

Can you open the code editor and write this in Python?

Python

```
from collections import deque

queue = deque()
print(queue)
```

Run it (you don’t have to submit yet), and tell me:

did it print something like deque([])?

Output

```
deque([])
```

yes

Nice, perfect start ☑ deque([]) means you’ve got an empty queue ready.

Now let’s add people to the line using append at the back.

Step by step:

Start with your empty queue = deque()
Do:

```
queue.append("Alice")
queue.append("Bob")
queue.append("Charlie")
```

Then print(queue)

Visually, after those appends, we want:

Text

```
front → [ 'Alice', 'Bob', 'Charlie' ] → back
```

In the code editor, update your code to:

Python

```
from collections import deque

queue = deque()

queue.append("Alice")
queue.append("Bob")
queue.append("Charlie")

print(queue)
```

Run it (no need to submit yet) and tell me what it prints.

```
deque(['Alice', 'Bob', 'Charlie'])
```

(fruit isnt in ou r code so its not called that in print)

Exactly right on both points 👌

Your queue output is perfect:

```
deque(['Alice', 'Bob', 'Charlie'])
```

That means new items are being added to the back with append, just like a real line.

You’re also right about the name: since we used "Alice", "Bob", "Charlie" in the code, that’s what shows up — no fruits here 👏😄

Now let’s do the queue part: removing from the front using popleft, to keep FIFO.

Concept first:

append → adds to the back
popleft → removes from the front (the one who has been waiting the longest)

Visually:

Text

start: front → ['Alice', 'Bob', 'Charlie'] → back

popleft() → removes 'Alice'
now: front → ['Bob', 'Charlie'] → back
Next step:

In the code editor, starting from your current code, add these lines at the end:

Python

```
first = queue.popleft()
print("Removed:", first)
print("Queue now:", queue)
```

Run it (you can tell me the output), and we'll check together that the behavior is truly FIFO.

```
deque(['Alice', 'Bob', 'Charlie'])
Removed: Alice
Queue now: deque(['Bob', 'Charlie'])
```

Beautiful. This is exactly how a FIFO queue should behave 🎯

Quick check:

Before: deque(['Alice', 'Bob', 'Charlie'])
After popleft():
Removed: Alice ☑ (the one who arrived first)
Queue now: deque(['Bob', 'Charlie']) ☑

So with deque:

Enqueue = queue.append(item) (goes to the back)
Dequeue = queue.popleft() (comes from the front)
Let's do one small, real-world style exercise to lock it in:

Imagine 3 customers arrive at a help desk in this order:

"Customer 1"
"Customer 2"
"Customer 3"

Then you serve 2 customers (so you call popleft() twice).

Step by step, in the code editor, write Python that:

Creates an empty deque called queue
Appends those three customers in order
Prints the queue
Does popleft() once, prints:
who was removed
the queue
Does popleft() a second time, prints:
who was removed
the final queue

When you're happy with it, hit Submit code so I can review your solution. 🚀

Python

```
from collections import deque

queue = deque()

queue.append("customer1")
queue.append("customer2")
queue.append("customer3")
print(queue)
first = queue.popleft()
print("Removed:", first)
print("Queue now:", queue)
first = queue.popleft()
print("Removed:", first)
print("Queue now:", queue)
```

Nice work, Ron — that's a clean queue implementation ☑

Your code:

Enqueues in order:
customer1 → customer2 → customer3
Then dequeues twice using popleft():
First removed: customer1
Second removed: customer2
Final queue: deque(['customer3'])

That's exactly FIFO using deque.append + deque.popleft 🎯

One small improvement for readability (not required, just style):

You might name the variables to match what's happening:

Python

```
served = queue.popleft()
print("Served:", served)
```

But functionally, what you wrote is perfect.

Now, one last thing to check your understanding conceptually:

Imagine you have this queue:

Python

```
from collections import deque

queue = deque([1, 2, 3])
```

To remove the oldest element (the 1) in a FIFO way when using deque, which method is the correct one to call?

A) queue.pop()

B) queue.popleft()

You can just answer: A or B.

b

Correct: B → queue.popleft() 🎯

With deque:

append() adds to the back
popleft() removes from the front → this keeps your queue FIFO

pop() would remove from the back, which would break FIFO.

You've now:

Built a queue with deque
Used append to enqueue
Used popleft to dequeue in FIFO order

You've hit all the goals for this lesson. 💪

Are you ready to wrap up this lesson and move on to the next one?

Yes, let's move on

Awesome, Ron — you nailed this one 👊

Quick recap of what you can now do:

Use from collections import deque to build a queue
Enqueue with queue.append(item) → adds to the back
Dequeue with queue.popleft() → removes from the front
Confirm FIFO behavior by printing the queue as it changes
These are the exact tools you'll reuse a lot when we work with more complex algorithms and systems.

When you're ready, hit Next Lesson in the Learn tab, and we'll keep building on this.
If anything about deque or queues still feels even a bit fuzzy, tell me and we'll tighten it up before you go on.

```
from collections import deque

queue = deque()

queue.append("customer1")
queue.append("customer2")
queue.append("customer3")
print(queue)
first = queue.popleft()
print("Removed:", first)
print("Queue now:", queue)
first = queue.popleft()
print("Removed:", first)
print("Queue now:", queue)
```